

```

#include <Wire.h>
#include <MPU6050.h>

// --- 1. SENSOR OBJECTS ---
MPU6050 upperArm(0x68); // AD0 Pin -> GND
MPU6050 forearm(0x69); // AD0 Pin -> 5V (CRITICAL for address 0x69)

// --- 2. PIN ASSIGNMENTS ---
const int FSR_PIN = A0;
// Motor A: Flexion (Upper)
const int RPWM1 = 5, LPWM1 = 6, EN1 = 7;
// Motor B: Extension (Lower)
const int RPWM2 = 9, LPWM2 = 10, EN2 = 8;

// --- 3. KALMAN FILTER VARIABLES ---
float Q = 0.022; // Process noise
float R = 0.15; // Measurement noise (Increase if jittery)
float P = 1.0, K = 0.0, filteredAngle = 0.0;

// --- 4. PARAMETERS ---
float homeAngle = 0;
const float targetLimit = 60.0; // Hard stop at 60 degrees
const float deadzone = 2.5; // Prevents motor jitter at 0 and 60
int fsrThreshold = 300; // Pressure to start assistance

int flexSpeed = 95; // Speed during flexion
int returnSpeed = 70; // Slower speed for safety return

void setup() {
  Wire.begin();
  Serial.begin(9600); // Stable baud for EMI suppression

  pinMode(RPWM1, OUTPUT); pinMode(LPWM1, OUTPUT); pinMode(EN1, OUTPUT);
  pinMode(RPWM2, OUTPUT); pinMode(LPWM2, OUTPUT); pinMode(EN2, OUTPUT);

  upperArm.initialize();
  forearm.initialize();

  Serial.println("SYSTEM INITIALIZING: Hold arm steady at 0 degrees...");
  delay(3000);

  // Auto-Calibration
  float sum = 0;
  for(int i=0; i<50; i++) { sum += getRawAngle(); delay(20); }

```

```

homeAngle = sum / 50.0;
filteredAngle = 0; // Initialize filter at rest

stopAll();
Serial.println("READY: Squeeze FSR to begin.");
}

void loop() {
  int pressure = analogRead(FSR_PIN);
  float rawDiff = abs(getRawAngle() - homeAngle);

  // --- APPLY KALMAN FILTER ---
  P = P + Q;
  K = P / (P + R);
  filteredAngle = filteredAngle + K * (rawDiff - filteredAngle);
  P = (1 - K) * P;

  // --- SERIAL DEBUGGING ---
  static unsigned long lastLog = 0;
  if (millis() - lastLog > 200) {
    Serial.print("Angle:"); Serial.print(filteredAngle);
    Serial.print(" | FSR:"); Serial.println(pressure);
    lastLog = millis();
  }

  // --- STATE MACHINE LOGIC ---

  // FLEXION STATE: FSR pressed & angle < 60
  if (pressure > fsrThreshold && filteredAngle < (targetLimit - deadzone)) {
    actuateFlexion();
  }

  // RETURN STATE: FSR released & angle > 0
  else if (pressure < fsrThreshold && filteredAngle > deadzone) {
    actuateExtension();
  }

  // IDLE STATE: Limit reached or at rest
  else {
    stopAll();
  }

  delay(15); // Loop stability
}

// --- HELPER FUNCTIONS ---

```

```

float getRawAngle() {
    int16_t ax1, ay1, az1, ax2, ay2, az2;
    upperArm.getAcceleration(&ax1, &ay1, &az1);
    forearm.getAcceleration(&ax2, &ay2, &az2);

    float p1 = atan2(ax1, sqrt(pow(ay1, 2) + pow(az1, 2))) * 180 / M_PI;
    float p2 = atan2(ax2, sqrt(pow(ay2, 2) + pow(az2, 2))) * 180 / M_PI;

    return abs(p1 - p2);
}

void actuateFlexion() {
    digitalWrite(EN1, HIGH); digitalWrite(EN2, HIGH);
    // Antagonistic: Motor A winds (Flex), Motor B unwinds (Slack)
    analogWrite(RPWM1, flexSpeed); analogWrite(LPWM1, 0);
    analogWrite(RPWM2, 0);      analogWrite(LPWM2, flexSpeed);
}

void actuateExtension() {
    digitalWrite(EN1, HIGH); digitalWrite(EN2, HIGH);
    // Reverse: Motor B winds (Return), Motor A unwinds (Slack)
    analogWrite(RPWM1, 0);      analogWrite(LPWM1, returnSpeed);
    analogWrite(RPWM2, returnSpeed); analogWrite(LPWM2, 0);
}

void stopAll() {
    digitalWrite(EN1, LOW); digitalWrite(EN2, LOW);
    analogWrite(RPWM1, 0); analogWrite(LPWM1, 0);
    analogWrite(RPWM2, 0); analogWrite(LPWM2, 0);
}

```